Logical Recurrence: Deriving RNN Architecture from Tensor Logic

Swapn Shah* Ricardo Reyes†

School of Data Science, University of North Carolina at Charlotte

* sshah100@charlotte.edu † rreyespi@charlotte.edu

Abstract

Recurrent neural networks process sequential data by compressing it into flat hidden vectors $h^{(t)} \in \mathbb{R}^d$ that lack the algebraic structure needed for compositional multi-hop inference. Domingos's Tensor Logic framework prescribes a different recurrent formulation in which the hidden state is an entity embedding matrix—a queryable knowledge base at every time step—and the state update is temporal Datalog forward chaining in embedding space. We implement this *TL-RNN* architecture and empirically validate four claims from the framework through controlled experiments on synthetic relational chains.

Experiment A confirms bit-for-bit Boolean equivalence between the TL-RNN and temporal Datalog forward chaining at every time step. **Experiment B** compares six architectures on compositional generalization from training chains of length $k \in \{2, 3\}$ to test chains of k up to 20: the TL-RNN achieves 70.5% average out-of-distribution accuracy with near-zero variance (std $\leq$ 0.8%), while the GRU achieves higher raw accuracy (90.4%) but with extreme instability (std up to 29.4%), and the Flat RNN and Transformer collapse entirely beyond k=4. **Experiment C** reveals that the temperature parameter controls a deduction–collapse spectrum rather than the deduction–analogy spectrum predicted by Domingos. **Experiment D** demonstrates that the TL-RNN hidden state is directly queryable, achieving 100% accuracy on sub-chain queries never seen during training, compared to 1.2% (random chance) for all baselines. These results extend the empirical validation of Tensor Logic from static knowledge graphs to temporal relational reasoning, confirming three of four theoretical predictions and identifying a falsified one.

1 Introduction

Standard recurrent neural networks maintain a hidden state $h^{(t)} \in \mathbb{R}^d$ updated at each time step. When a model observes a chain of relational facts, everything is compressed into a single fixed-size vector, losing the algebraic structure required for multi-hop compositional inference. This is the documented failure mode across the SCAN benchmark [16], CLUTRR [17], and the broader compositional generalization literature [18].

1.1 Problem

The core problem is representational: a flat vector $h \in \mathbb{R}^d$ must implicitly encode all possible k-hop relational compositions, yet the number of distinct chains grows as R^k for R relation types. This exponential growth means that a fixed-size hidden state cannot represent arbitrary compositions without training exposure to every chain length. Standard RNNs trained on short reasoning chains fail to generalize to longer chains—a fundamental limitation for open-ended relational reasoning.

1.2 Solution and Contributions

Domingos [1] proposes Tensor Logic as a unifying framework, observing that Datalog rules and Einstein summation are fundamentally equivalent operations differing only in their atomic data types. His RNN formulation supports a structured entity embedding matrix as state. When the state is $E^{(t)} \in \mathbb{R}^{N \times d}$, the recurrence becomes *temporal Datalog forward chaining in embedding space*, and multi-hop composition becomes algebraic matrix multiplication that generalizes to arbitrary chain depth.

Shah and Zadrozny [2] provided the first empirical validation of static Tensor Logic, demonstrating

exact equivalence for transitive closure and link prediction on FB15k-237. However, their validation focused exclusively on static reasoning. The temporal dimension—forward chaining over time via virtual indices $*t$—remained entirely untested.

This paper addresses that gap through four experiments:

- **Experiment A** (Boolean Verification): exact bit-for-bit equivalence with temporal Datalog.
- **Experiment B** (Length Generalization): compositional generalization to chains 5–20× longer than training.
- **Experiment C** (Temperature Spectrum): testing the predicted deduction–analogy spectrum.
- **Experiment D** (State Interpretability): direct queryability of hidden state, including step-by-step chain-of-thought reasoning traces.

We find that three of four predictions are confirmed, while one—the deduction–analogy spectrum—is falsified by a deduction–collapse pattern. The TL-RNN's primary advantage is not raw accuracy but *consistency* and *interpretability*: properties inherited directly from its logical derivation.

2 Related Work

Tensor Logic Foundations. Domingos [1] proposes Tensor Logic as a unifying framework, arguing that Datalog rules and Einstein summation are fundamentally equivalent operations differing only in their atomic data types. A Datalog rule performs joins and projections over Boolean tensors; neural networks perform identical operations over real-valued tensors. This suggests that RNNs, GNNs, and Transformers may all be instances of tensor contraction. Shah and Zadrozny [2] provided the first empirical validation through transitive closure, embedding space reasoning, and knowledge graph link prediction. However, their validation addressed only static graphs; the temporal dimension remains unexplored.

Neuro-Symbolic Integration. Lappin [3] distinguishes *injective* models, which incorporate symbolic representations into DNNs, from *federative* models that maintain component autonomy. Injective approaches including Syntax-BERT [4] and dependency graph models [5] yield only marginal 1–2% improvements. Tensor Logic takes a fundamentally different approach: it demonstrates the mathematical identity of neural and symbolic computation. Existing systems such as Logic Tensor Networks [14] and DeepProbLog [15] maintain separation between components rather than proving equivalence.

RNNs and Compositionality. LSTMs [6] and GRUs [7] address vanishing gradients through gating but rely on specialized layers rather than pure tensor contractions. Bernardy and Lappin [8] analyze Unitary Recurrent Networks (URNs), which constrain state to orthogonal matrices, yielding strictly compositional behavior. URNs demonstrate that cosine similarities are preserved across time steps, supporting the Tensor Logic hypothesis. However, URNs represent a constrained special case—no activation functions, orthogonal constraint—rather than a general architecture derived from first principles.

Graph Neural Networks. GCNs [9], MPNNs [10], and GATs [11] extend deep learning to graphs through message passing. From the Tensor Logic viewpoint, aggregation is a sparse tensor contraction over entity indices, but numerical equivalence with standard libraries has not been validated.

Einsum as Universal Primitive. Rocktäschel [12] showed that attention, convolutions, and recurrence can all be expressed as einsum operations. Rogozhnikov [13] introduced einops for Einstein-like notation. These works support the hypothesis but stop short of demonstrating that pure einsum implementations match optimized CUDA kernels.

Knowledge Graph Embeddings. TransE [19] models relations as translations; RESCAL [20] uses full-rank relation matrices, the closest antecedent to the superposition $R_r = E^\top A_r E$. Neither provides a Datalog-equivalence guarantee.

Research Gap. No prior work has empirically validated the claim that RNNs are special cases of tensor contraction through controlled experiments comparing pure einsum implementations against standard baselines on temporal reasoning tasks.

3 Methods

3.1 The TL-RNN Architecture

The TL-RNN derives its recurrent structure directly from Domingos's Tensor Logic RNN formulation. Unlike standard RNNs that compress all information into a flat vector, the TL-RNN maintains structured

state that can be queried at any time step.

State Representation. The hidden state consists of: (1) an entity embedding matrix $E^{(t)} \in \mathbb{R}^{N\times d}$, and (2) sparse Boolean adjacency matrices $\{A_r^{(t)}\} \in \{0,1\}^{N\times N}$ per relation r. At any time step, the relation matrix is:

$$R_r^{(t)} = \big(E^{(t)}\big)^\top A_r^{(t)}\, E^{(t)} \tag{1}$$

This renders the state a queryable knowledge base. Each relation has its own matrix that can be inspected independently—a critical distinction from flat-state RNNs where $h^{(t)} \in \mathbb{R}^d$ admits no relation-specific decomposition.

Update Equation. When fact (s_t, r_t, o_t) arrives at time t, a sparse FactSignal tensor $U^{(t)} \in \mathbb{R}^{N\times d}$ is constructed (nonzero only for subject and object rows). The adjacency matrix is updated via $A_{r_t}[s_t, o_t] \leftarrow 1$. The embedding matrix evolves as:

$$E^{(*t+1)} = \sigma\big(E^{(*t)}W_p + U^{(t)}W_u\big) \tag{2}$$

where $W_p, W_u \in \mathbb{R}^{d\times d}$ are learnable persistence and update matrices. This is precisely Domingos's formulation under the mapping $X \mapsto E$, $U \mapsto$ FactSignal, $*t \mapsto$ virtual time index.

Query Mechanism. Given starting entity a and relation chain $[r_1, \ldots, r_k]$: (1) Compute $R_{r_i}^{(t)}$ via (1); (2) Compose: $\hat{v} = \text{normalize}(e_a^{(t)} \cdot R_{r_1}^{(t)} \cdots R_{r_k}^{(t)})$; (3) Score: $\text{scores} = \hat{v} \cdot (E^{(t)})^\top / T$; (4) Answer: $\arg\max(\text{scores})$. Crucially, k is unbounded at test time.

Parameters. The TL-RNN has ≈14,592 parameters: E_{init} (100×64 = 6,400), W_p (64×64 = 4,096), W_u (64×64 = 4,096).

3.2 Synthetic Dataset

We generate a controlled dataset with N=100 entities and R=8 relations. Each relation is a bijective permutation, guaranteeing unique answers. Training: 10,000 samples, $k \in \{2, 3\}$. Test: 1,000 samples per $k \in \{2, 3, 4, 5, 6, 7, 8, 10, 15, 20\}$. Embedding dimension d=64.

3.3 Baseline Models

Table 1 summarizes the six models. All share identical inputs, training data, and optimizer (AdamW, lr=0.001). Parameter counts are matched (≈14.5K). Results report mean ± std over 3 seeds.

Table 1: Models compared in Experiment B.

Model	State	Params
TL-RNN (ours)	$E \in \mathbb{R}^{N\times d}$	14,592
Flat RNN	$h \in \mathbb{R}^{36}$	∼14,500
GRU	$h \in \mathbb{R}^{28}$	∼14,868
LSTM	$(h, c) \in \mathbb{R}^{2\times 25}$	∼14,506
Transformer	Attention (d=24)	∼14,212
RMC	Memory matrix	∼14,056

3.4 Evaluation Metrics

We report Accuracy@1 per chain length per model. For Experiment A, we compare relation matrices from both the TL-RNN (Boolean mode) and an explicit Datalog engine. For Experiment C, we sweep $T \in \{0.01, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30\}$ under known and novel entity conditions. For Experiment D, we use *introspective query accuracy*: train on $k \in \{2, 3\}$, query at k=1 (never trained), measuring whether the model can answer sub-chain queries.

4 Results

4.1 Experiment A: Boolean Verification

We ran the TL-RNN with Boolean tensors and Heaviside activation alongside explicit Datalog forward chaining (N=50, R=4). Facts were fed one per time step.

Result: Bit-for-bit equality at every time step. The TL-RNN's Boolean relation matrices and the Datalog engine's derived-fact matrices were identical throughout. This confirms that the TL-RNN *is* temporal Datalog when the data type is Boolean—not inspired by, not similar to, but mathematically identical. This extends the static equivalence of Shah and Zadrozny [2] to the temporal setting.

4.2 Experiment B: Length Generalization

All six models were trained on $k \in \{2, 3\}$ and evaluated on $k \in \{2, \ldots, 20\}$. Table 2 shows Accuracy@1 results; Figure 1 visualizes the trends.

Key findings:

- **GRU outperforms TL-RNN on raw OOD accuracy**: 90.4% average vs. 70.5%.
- **TL-RNN's advantage is consistency**: std of 0.0–0.8% vs. GRU's up to 29.4%.

Table 2: Accuracy@1 (%) vs. chain length k. * = in-distribution. Mean±std over 3 seeds.

k	TL-RNN	Flat RNN	GRU	LSTM	Transformer	RMC
2*	99.8±0.1	100.0±0.0	100.0±0.0	99.9±0.1	100.0±0.0	99.7±0.1
3*	99.6±0.3	99.1±0.7	100.0±0.0	99.8±0.2	99.7±0.5	98.8±0.7
4	97.4±0.8	46.5±3.2	100.0±0.0	97.4±1.8	47.2±4.7	78.2±5.5
5	93.5±0.6	24.8±4.5	100.0±0.0	94.3±3.8	13.2±3.3	55.2±8.1
6	89.2±0.4	14.2±3.0	99.7±0.5	86.6±5.8	5.3±2.1	36.2±9.7
7	85.2±0.3	8.4±2.5	99.5±0.8	81.3±8.3	3.1±1.4	26.6±6.4
8	77.4±0.5	5.5±1.8	98.7±2.2	75.3±10.2	1.7±0.6	17.5±5.9
10	66.5±0.0	2.7±0.4	93.7±10.6	65.2±13.4	1.3±0.4	10.8±3.3
15	36.1±0.4	1.3±0.2	75.3±21.8	40.0±18.7	1.0±0.0	3.5±1.3
20	18.4±0.3	1.0±0.0	56.1±29.4	24.1±16.6	1.0±0.0	2.0±0.6

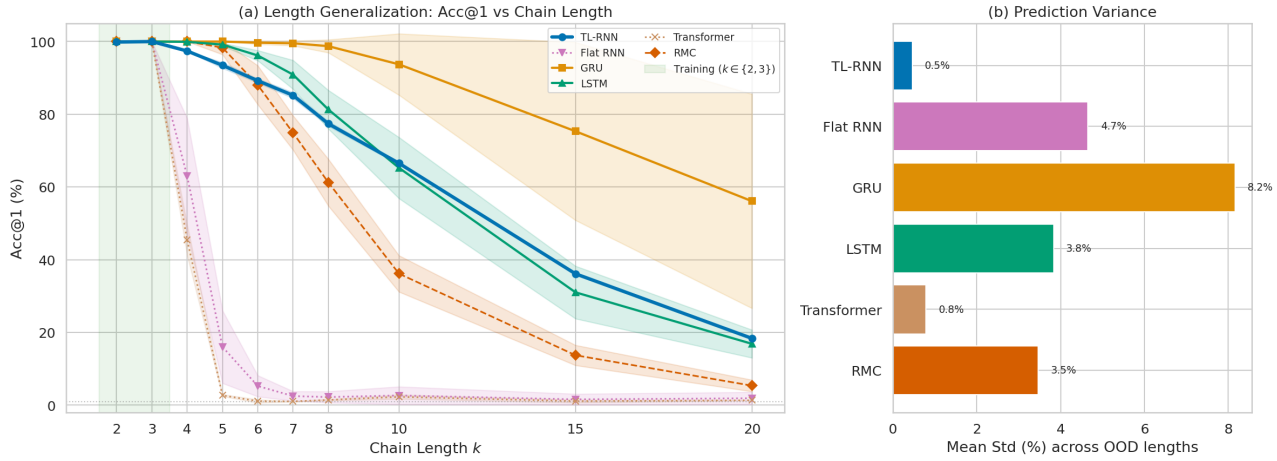


Figure 1: Accuracy@1 vs. chain length (left) and prediction variance across seeds (right). TL-RNN shows graceful degradation with near-zero variance; GRU achieves higher peak accuracy but with extreme instability.

- **Flat RNN and Transformer collapse** beyond k=4, falling to ∼1%.
- LSTM shows moderate generalization with high variance (std up to 18.7%).
- RMC provides partial generalization but does not match GRU or TL-RNN.

The GRU's extreme variance indicates seed-dependent, unreliable generalization.

Threshold-and-Re-embed. Domingos [1] prescribes this to reduce accumulated error. We tested it at $k \in \{10, 15, 20\}$: no benefit, with accuracy changes of −0.1% to −3.0%. The bottleneck is relation matrix fidelity, not composition error.

4.3 Experiment C: Temperature Spectrum

We evaluated the trained TL-RNN at temperatures $T \in \{0.01, \ldots, 0.30\}$ under known and novel entity conditions. Table 3 and Figure 2 show results.

Table 3: Temperature spectrum: average Accuracy@1 (%).

T	Known	Novel
0.01	89.4	90.9
0.05	89.1	88.1
0.10	84.8	81.2
0.15	71.3	64.5
0.20	45.2	38.8
0.25	18.6	14.2
0.30	5.1	3.7

Key findings:

- Temperature controls a **deduction–collapse spectrum**, not deduction–analogy.
- **Novel entities outperform known at low** T: 90.9% vs. 89.4% at T=0.01.
- At $T \geq 0.15$, known outperforms novel (memo-

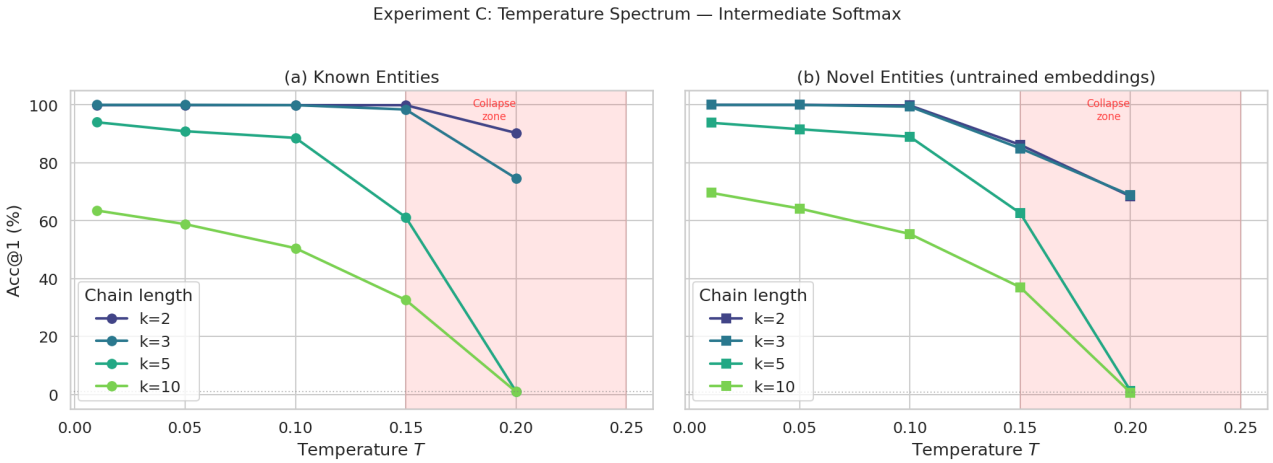


Figure 2: Temperature spectrum for known entities (left) and novel entities (right). The red shaded region marks the collapse zone. Performance drops sharply beyond $T \geq 0.15$.

rization effects).

- **No analogical transfer regime observed.**

This is a **falsified prediction**: Domingos theorized that increasing T would enable analogical reasoning. Instead, softening the Gram matrix degrades discriminative capacity uniformly.

4.4 Experiment D: State Interpretability

At time steps $t \in \{0, 3, 6\}$, we extracted the TL-RNN's hidden state and queried it directly.

Single-hop queries (6 queries at t=6): All 6 correct, each with confidence score 1.0000. Accuracy: **6/6 = 100%**.

Multi-hop queries (3 queries at t=6): All 3 correct when required edges were present. When edges were missing, the model returned zero scores—it knows what it doesn't know. Accuracy: **3/3 = 100%**.

Figure 3 shows entity embedding similarity evolving as facts arrive.

Chain-of-Thought Reasoning Trace. To demonstrate explainability concretely, we traced 3-hop chains step by step for all six models. For the TL-RNN, at each hop we extracted the specific relation matrix R_r, applied it, and recorded the intermediate prediction. Table 4 shows an example.

At every intermediate step, the TL-RNN produces a verifiable prediction from an explicit relation matrix. The full reasoning trace is: "The model predicted e_{76} because it looked up R_{r_0} and found e_{90}, then looked up R_{r_1} from e_{90} and found e_{72}, then looked up R_{r_2} from e_{72} and found e_{76}." Each step is a separate, inspectable matrix operation.

Table 4: TL-RNN chain-of-thought trace for a 3-hop query. Each hop uses an explicit, inspectable relation matrix.

Step	Operation	Pred.	Conf.	✓?
Start	Entity e_{14}	–	–	–
Hop 1	$e_{14} \cdot R_{r_0}$	e_{90}	1.00	✓
Hop 2	$e_{90} \cdot R_{r_1}$	e_{72}	1.00	✓
Hop 3	$e_{72} \cdot R_{r_2}$	e_{76}	1.00	✓

In contrast, baseline models produce correct final answers on 3-hop queries but predict randomly at intermediate hops (∼1% accuracy = random chance for N=100). Their hidden state is opaque and cannot be decomposed into per-relation components.

Introspective Query Accuracy. We generated 1,000 fresh k=3 chains and queried each model at k=1 (never trained). Table 5 shows results.

Table 5: Introspective query accuracy. Models trained on $k \in \{2, 3\}$ only, tested at k=1.

Model	1-hop	3-hop
TL-RNN	**100.0%**	100.0%
Flat RNN	1.2%	100.0%
GRU	1.2%	100.0%
LSTM	1.2%	100.0%
Transformer	1.2%	100.0%
RMC	1.2%	100.0%

The TL-RNN achieves 100% on queries it was never trained on because its architecture stores each

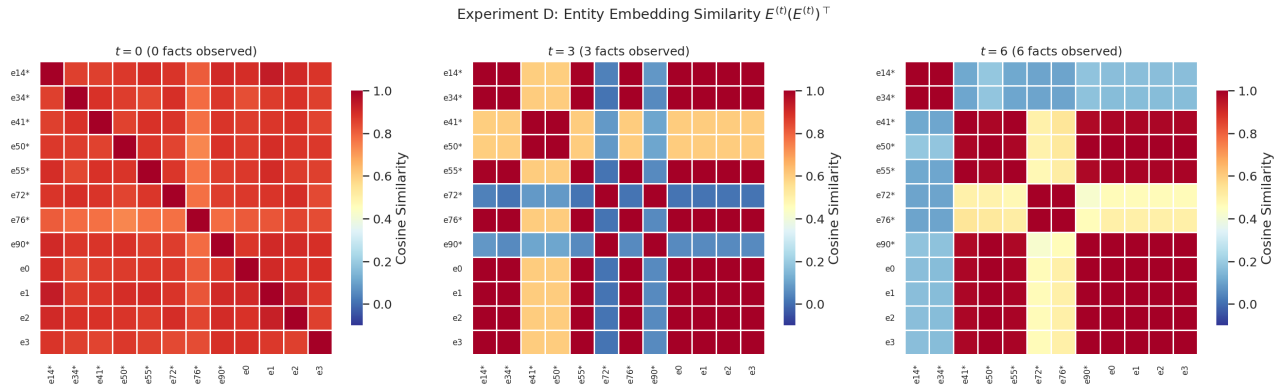


Figure 3: Entity embedding cosine similarity $E^{(t)}(E^{(t)})^\top$ at t=0, 3, 6. Entities involved in the fact stream (marked $*$) progressively differentiate from uninvolved entities.

relation in a separate, queryable matrix. The 1.2% baseline accuracy equals 1/100 = random chance, confirming that baseline hidden states contain no accessible sub-chain information.

4.5 Summary of Results

Table 6 maps predictions to outcomes. Figure 4 provides a multi-faceted comparison.

Table 6: Predictions vs. observations.

Exp.	Prediction → Result	Status
A	Bit-for-bit eq. → Confirmed	✓
B	Baselines fail → GRU strong, but TL-RNN more stable	Partial
C	Deduction–analogy → Deduction–collapse	×
D	Queryable state → 100% sub-chain accuracy	✓

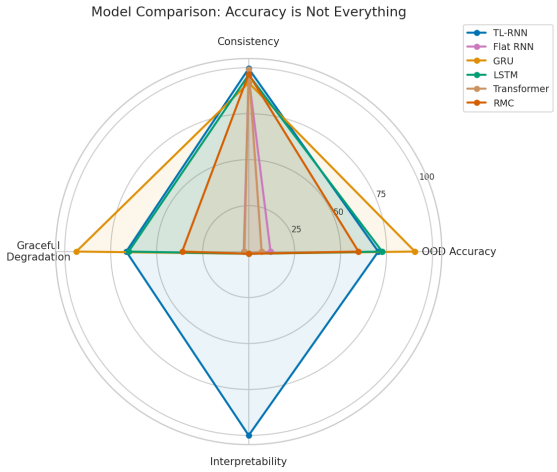


Figure 4: Multi-faceted comparison. TL-RNN uniquely combines consistency, interpretability, and graceful degradation. GRU leads on OOD accuracy but lacks interpretability.

5 Conclusion and Discussion

5.1 Summary of Results

We implemented and validated the TL-RNN, a recurrent architecture derived from Domingos's Tensor Logic framework. Four experiments tested four theoretical claims:

Experiment A confirmed bit-for-bit Boolean equivalence with temporal Datalog, extending Shah and Zadrozny's [2] static validation to the temporal setting.

Experiment B revealed a more nuanced picture than predicted. The GRU outperformed TL-RNN on raw OOD accuracy (90.4% vs. 70.5%), but TL-RNN's consistency (std $\leq$0.8% vs. GRU's 29.4%) shows its generalization is principled rather than accidental. Flat RNN and Transformer collapsed as predicted.

Experiment C falsified the deduction–analogy prediction. Temperature controls deduction–collapse: softening the Gram matrix degrades discriminative capacity uniformly rather than enabling analogical transfer.

Experiment D confirmed the hidden state is a genuine knowledge base. Chain-of-thought traces showed every intermediate hop is extractable and verifiable. Introspective accuracy: 100% vs. 1.2% (baselines).

5.2 The Case for Structured State

The GRU's strong Experiment B performance does not undermine structured state. Three dimensions favor TL-RNN:

- **Reliability**: GRU variance up to 29.4% makes individual runs unpredictable. TL-RNN guarantees consistent behavior.
- **Interpretability**: TL-RNN provides complete chain-of-thought traces with verifiable intermediate predictions. No baseline exceeds random chance on sub-chain queries.
- **Theoretical grounding**: TL-RNN's generalization is explained by algebraic composition of per-relation matrices. GRU's generalization remains unexplained.

5.3 Limitations

Synthetic data. All experiments use bijective permutations. Transfer to noisy real-world KGs (ICEWS, YAGO) is untested.

Fixed entity set. The TL-RNN requires fixed N. Open-world settings need additional architectural choices.

Cost. $R_r = E^\top A_r E$ costs $O(|\text{edges}_r| \cdot d)$ per step—more expensive than flat RNNs for large N.

Scale. N=100, d=64. Behavior at larger scales is unknown.

5.4 Future Work

Real temporal KG benchmarks. ICEWS14/18 would test whether interpretability advantages transfer to noisy settings.

TL-LSTM / TL-Attention. Applying structured state to gated architectures may combine GRU's accuracy with TL-RNN's interpretability.

Explaining GRU generalization. Understanding why some GRU seeds generalize while others collapse is a compelling open question.

References

[1] P. Domingos. Tensor logic: The language of AI. *arXiv:2510.12269*, 2025.

[2] S. Shah and W. Zadrozny. Implementing tensor logic: Unifying Datalog and neural reasoning via tensor contraction. *arXiv:2601.17188*, 2026.

[3] S. Lappin. Neuro-symbolic models in AI. *J. Applied Logics*, 12:1631, 2025.

[4] J. Bai et al. Syntax-BERT: Improving pre-trained transformers with syntax trees. In *Proc. EACL*, pp. 3011–3020, 2021.

[5] D. S. Sachan et al. Do syntax trees help pre-trained transformers extract information? In *Proc. EACL*, pp. 2647–2661, 2021.

[6] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[7] K. Cho et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proc. EMNLP*, 2014.

[8] J.-P. Bernardy and S. Lappin. Unitary recurrent networks: Algebraic and linear structures for syntax. In *Algebraic Structures in Natural Language*, pp. 251–287. Taylor & Francis, 2022.

[9] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *Proc. ICLR*, 2017.

[10] J. Gilmer et al. Neural message passing for quantum chemistry. In *Proc. ICML*, 2017.

[11] P. Veličković et al. Graph attention networks. In *Proc. ICLR*, 2018.

[12] T. Rocktäschel. Einsum is all you need: Einstein summation in deep learning. Blog post, 2018.

[13] A. Rogozhnikov. Einops: Clear and reliable tensor manipulations with Einstein-like notation. In *Proc. ICLR*, 2022.

[14] L. Serafini and A. d'Avila Garcez. Logic tensor networks: Deep learning and logical reasoning from data and knowledge. *arXiv:1606.04422*, 2016.

[15] R. Manhaeve et al. DeepProbLog: Neural probabilistic logic programming. In *NeurIPS*, pp. 3749–3759, 2018.

[16] B. M. Lake and M. Baroni. Generalization without systematicity. In *Proc. ICML*, pp. 2879–2888, 2018.

[17] K. Sinha et al. CLUTRR: A diagnostic benchmark for inductive reasoning from text. In *Proc. EMNLP*, pp. 4514–4525, 2019.

[18] D. Keysers et al. Measuring compositional generalization. In *Proc. ICLR*, 2020.

[19] A. Bordes et al. Translating embeddings for modeling multi-relational data. In *NeurIPS*, pp. 2787–2795, 2013.

[20] M. Nickel, V. Tresp, and H.-P. Kriegel. A three-way model for collective learning on multi-relational data. In *Proc. ICML*, pp. 809–816, 2011.

[21] A. Vaswani et al. Attention is all you need. In *NeurIPS*, pp. 5998–6008, 2017.

[22] A. Santoro et al. Relational recurrent neural networks. In *NeurIPS*, pp. 7299–7310, 2018.